

AI READY PREMIUM KIT

Python come primo passo verso Dati, Automazione e Intelligenza Artificiale

Guida pratica in 12 capitoli, con esempi, esercizi e checklist

Materiale bonus di "Python da Zero con Google Colab"
Tania Cilio

Indice

Come usare questa guida

Questa guida è il ponte tra le basi di Python che hai imparato nel libro e i tre campi che stanno cambiando il mondo del lavoro: i dati, l'automazione e l'intelligenza artificiale. Non insegna cose nuove e complicate: ti mostra come usare quello che già sai per fare cose vere e utili.

È organizzata in 12 capitoli a difficoltà crescente. Ogni capitolo ha la stessa struttura: una spiegazione chiara, un esempio pratico con codice reale che puoi eseguire in Google Colab, un mini esercizio per provare da solo, e una checklist per verificare di aver capito prima di andare avanti.

Consiglio d'uso: non limitarti a leggere. Apri Google Colab di fianco, esegui ogni esempio, prova a modificarlo, e fai i mini esercizi. La differenza tra chi "ha letto" e chi "sa fare" è tutta qui. Tutti gli esempi di questa guida sono stati eseguiti e verificati.

Capitolo 1

Perché Python è il ponte verso dati, automazione e AI

Hai imparato le basi di Python. Forse ti stai chiedendo: e adesso? A cosa serve davvero tutto questo? Questa guida risponde proprio a questa domanda. Le basi che conosci non sono un punto di arrivo: sono un ponte. E questo capitolo ti mostra dove porta quel ponte.

Python è oggi il linguaggio più usato al mondo in tre campi che stanno cambiando il lavoro di tutti: l'analisi dei dati, l'automazione dei compiti ripetitivi e l'intelligenza artificiale. Non è un caso. Python è nato per essere leggibile e semplice, e questa semplicità lo ha reso lo strumento preferito di chi lavora con i dati e con l'AI.

La cosa importante da capire è questa: per fare quei lavori non servono superpoteri, servono le basi solide più qualche strumento in più. Le variabili, le liste, i dizionari, i cicli e le funzioni che già conosci sono esattamente i mattoni con cui si costruiscono analisi di dati e automazioni. Non butterai via niente di quello che hai imparato: lo userai tutto.

Facciamo un esempio concreto di cosa intendiamo. Immagina di avere un file con le vendite di un negozio: 500 righe, ognuna con prodotto, quantità e prezzo. A mano, calcolare il totale o trovare il prodotto più venduto richiede ore e si rischiano errori. Con Python sono poche righe di codice, e il risultato è immediato e sempre corretto. Questo è il potere che stai per sbloccare.

In questa guida faremo un percorso preciso: prima ripasseremo le basi che ci servono, poi impareremo a trattare i dati come tabelle, a leggerli da file veri, a pulirli, ad analizzarli, e infine a costruire piccole automazioni e a capire come Python si collega all'intelligenza artificiale. Alla fine avrai costruito, passo dopo passo, un vero mini-strumento di analisi.

Esempio pratico

```
# Un assaggio: da una lista di vendite al totale, in 4 righe
vendite = [120.50, 89.00, 230.75, 45.20, 99.99]
totale = sum(vendite)
media = totale / len(vendite)
print(f"Totale vendite: {totale:.2f} euro")
print(f"Vendita media: {media:.2f} euro")
print(f"Vendita più alta: {max(vendite):.2f} euro")
```

Guarda quanto è compatto. Con `sum()`, `len()` e `max()` - funzioni che già conosci - hai già fatto una piccola analisi di dati. Questo è il punto di partenza: niente di nuovo, solo le basi usate con uno scopo.

Mini esercizio

Crea una lista con i prezzi di 6 prodotti che compri di solito. Stampa il totale, la media (con 2 decimali) e il prezzo più basso. Usa `sum()`, `len()` e `min()`.

Checklist: hai imparato a...

- ☐ Ho capito che le basi di Python sono il punto di partenza, non di arrivo.
- ☐ So che dati, automazione e AI usano gli stessi mattoni che già conosco.
- ☐ Ho eseguito l'esempio e ottenuto i tre valori.
- ☐ Ho fatto il mini esercizio con i miei dati.

Capitolo 2

Ripasso essenziale: le basi che useremo sempre

Prima di costruire, controlliamo che le fondamenta siano solide. In questo capitolo ripassiamo velocemente i mattoni che useremo in tutta la guida. Se qualcosa non ti è chiaro, è il momento di tornare al capitolo corrispondente del libro: non andare avanti con dubbi sulle basi.

Le variabili sono contenitori con un nome. A sinistra il nome, a destra il valore. Ne esistono di diversi tipi: il testo (stringa), i numeri interi (int), i numeri decimali (float) e i valori vero/falso (bool). Saper distinguere i tipi è fondamentale quando si lavora con i dati, perché un numero scritto come testo non si può sommare.

Le f-string sono il modo moderno di costruire messaggi che contengono variabili. Si scrivono mettendo una f prima delle virgolette e racchiudendo le variabili tra graffe. Dentro le graffe puoi anche fare calcoli e formattare i numeri: per esempio `:.2f` arrotonda a due decimali, comodissimo per i soldi.

La conversione dei tipi è un'operazione che faremo spessissimo con i dati reali. Quando leggi un dato da un file o da una tastiera, arriva sempre come testo. Per usarlo come numero devi convertirlo: `int()` per gli interi, `float()` per i decimali. È una delle cause di errore più comuni per i principianti, quindi tienila a mente.

Esempio pratico

```
# Ripasso pratico dei tipi e delle conversioni
prodotto = "Caffè"           # stringa (testo)
quantita = 3                 # intero
prezzo = 1.20                # decimale
disponibile = True          # booleano

# Una f-string che mette tutto insieme e calcola il totale
totale = quantita * prezzo
print(f"{prodotto}: {quantita} pezzi x {prezzo:.2f} = {totale:.2f} euro")

# Conversione: un dato che arriva come testo va trasformato in numero
prezzo_testo = "2.50"
prezzo_numero = float(prezzo_testo)
print(f"Doppio del prezzo: {prezzo_numero * 2:.2f}")
```

Nota due cose. Primo: la f-string permette di unire testo, numeri e calcoli in una sola riga leggibile. Secondo: `prezzo_testo` era una stringa (tra virgolette) e l'abbiamo dovuto convertire con `float()` prima di poterlo moltiplicare. Senza la conversione, Python avrebbe dato errore.

Mini esercizio

Crea le variabili per un prodotto a tua scelta (nome, quantità, prezzo). Stampa una riga riassuntiva con la f-string e il totale formattato a 2 decimali. Poi prendi un prezzo scritto come testo (es. "9.99"), convertilo e stampane la metà.

Checklist: hai imparato a...

- ☐ So distinguere stringa, intero, decimale e booleano.
- ☐ So usare una f-string con variabili e calcoli dentro le graffe.
- ☐ So formattare un numero a 2 decimali con `:.2f`.
- ☐ So convertire un testo in numero con `int()` o `float()` e perché serve.

Capitolo 3

Pensare per dati: liste e dizionari come tabelle

Quando si lavora con i dati, la prima cosa da imparare è rappresentarli nel modo giusto. Python ci dà due strumenti perfetti che già conosci: le liste e i dizionari. Insieme, permettono di rappresentare qualsiasi tabella di dati.

Una lista è una sequenza ordinata di elementi: pensala come una colonna o come un elenco. Un dizionario associa delle etichette (chiavi) a dei valori: pensalo come una singola riga di una tabella, dove ogni colonna ha un nome. Per esempio, un cliente può essere un dizionario con chiavi nome, età e città.

La combinazione potente è questa: una lista di dizionari. Ogni dizionario è una riga, la lista è l'intera tabella. Questo è il modo più comune di rappresentare dati in Python prima di passare a strumenti più avanzati. Se capisci bene questa struttura, hai capito il 70% di come si trattano i dati.

Per leggere tutti i dati di una lista di dizionari si usa il ciclo for: scorri la lista, e per ogni dizionario (ogni riga) accedi ai valori con le loro chiavi. È lo schema che ripeteremo in quasi tutti i capitoli successivi, quindi vale la pena farlo proprio.

Esempio pratico

```
# Una tabella di clienti come lista di dizionari
clienti = [
    {"nome": "Anna", "città": "Roma", "spesa": 120.50},
    {"nome": "Marco", "città": "Milano", "spesa": 89.00},
    {"nome": "Sara", "città": "Roma", "spesa": 230.75},
]

# Scorro la tabella riga per riga
for cliente in clienti:
    print(f"{cliente['nome']} ({cliente['città']}): {cliente['spesa']:.2f} euro")

# Una piccola analisi: spesa totale di tutti i clienti
totale = 0
for cliente in clienti:
    totale += cliente["spesa"]
print(f"Spesa totale: {totale:.2f} euro")
```

Questa struttura - lista di dizionari - è il cuore del lavoro con i dati. Ogni dizionario è una riga con le sue colonne (nome, città, spesa). Il primo ciclo stampa la tabella; il secondo usa un accumulatore per sommare una colonna. Sono due operazioni che farai continuamente.

Mini esercizio

Crea una lista di almeno 4 dizionari che rappresentano dei libri (titolo, autore, prezzo). Stampa ogni libro su una riga, poi calcola e stampa il prezzo medio di tutti i libri.

Checklist: hai imparato a...

- ☐ Ho capito che un dizionario rappresenta una riga e una lista di dizionari una tabella.
- ☐ So scorrere una lista di dizionari con un ciclo for.
- ☐ So accedere ai valori di ogni dizionario con le chiavi.
- ☐ So calcolare un totale o una media scorrendo i dati.

Capitolo 4

Leggere e scrivere file di testo

Finora i dati erano scritti dentro il codice. Ma i dati veri vivono nei file. In questo capitolo impari a leggere e scrivere file di testo, il primo passo per lavorare con informazioni che esistono fuori dal tuo programma.

Per aprire un file si usa l'istruzione `with open()`. La parola `with` garantisce che il file venga chiuso correttamente quando hai finito, anche se qualcosa va storto: è una buona abitudine da prendere subito. Dentro `open()` indichi il nome del file e la modalità: `"r"` per leggere (read), `"w"` per scrivere (write, sovrascrive tutto), `"a"` per aggiungere in fondo (append).

Quando scrivi su un file, ricordati di andare a capo tu, aggiungendo il carattere speciale `backslash-n` alla fine di ogni riga. A differenza di `print()`, che va a capo da solo, la scrittura su file mette esattamente quello che le dici, niente di più.

Quando leggi un file riga per riga con un ciclo `for`, ogni riga arriva con il suo a-capo finale. Per pulirla si usa `.strip()`, che toglie spazi e a-capo ai bordi. È un passaggio quasi sempre necessario quando si leggono dati da file.

Esempio pratico

```
# Scrivo tre righe in un file
with open("note.txt", "w", encoding="utf-8") as file:
    file.write("Prima nota\n")
    file.write("Seconda nota\n")
    file.write("Terza nota\n")

# Rileggo il file riga per riga, pulendo ogni riga
with open("note.txt", "r", encoding="utf-8") as file:
    numero = 1
    for riga in file:
        print(f"{numero}. {riga.strip()}")
        numero += 1
```

Indica sempre `encoding="utf-8"`: serve a gestire correttamente le lettere accentate italiane. Nota il `backslash-n` alla fine di ogni `write()`: senza, le tre note finirebbero tutte attaccate sulla stessa riga. In lettura, `.strip()` toglie l'a-capo che altrimenti creerebbe righe vuote di troppo.

Mini esercizio

Scrivi in un file chiamato `spesa.txt` una lista di 5 prodotti, uno per riga. Poi rileggi il file e stampa ogni prodotto numerato (1, 2, 3...). Infine, riapri il file in modalità `append` e aggiungi un sesto prodotto.

Checklist: hai imparato a...

- ☐ So aprire un file con `with open()` e la modalità giusta (`r`, `w`, `a`).
- ☐ So scrivere righe ricordando l'a-capo `backslash-n`.
- ☐ So leggere un file riga per riga con un `for`.
- ☐ So pulire ogni riga letta con `.strip()` e perché serve.

Capitolo 5

I file CSV: il formato dei dati veri

Il CSV è il formato di dati più diffuso al mondo. La sigla significa Comma-Separated Values, cioè valori separati da virgola (in Italia spesso si usa il punto e virgola, per non confondersi con la virgola dei decimali). Ogni riga del file è una riga di tabella, e i campi sono separati dal separatore. Excel, i gestionali, i siti web: quasi tutto esporta in CSV.

La cosa bella è che un CSV è solo un file di testo. Quindi sai già leggerlo, con quello che hai imparato nel capitolo precedente. Per separare i campi di ogni riga si usa `.split()`, indicando il separatore: `riga.split(";")` restituisce una lista con i singoli valori.

Attenzione a un punto cruciale: i valori letti da un CSV sono sempre testo. Se una colonna contiene numeri (prezzi, quantità), devi convertirli con `int()` o `float()` prima di poterci fare calcoli. Dimenticarlo è l'errore numero uno quando si elaborano i CSV.

Di solito la prima riga di un CSV contiene le intestazioni (i nomi delle colonne). Spesso conviene saltarla quando elabori i dati veri. Vedremo come fare. Trasformare ogni riga del CSV in un dizionario, usando le intestazioni come chiavi, è il modo più pulito di lavorare: ritrovi la struttura lista-di-dizionari del capitolo 3.

Esempio pratico

```
# Simulo un piccolo CSV scrivendolo, poi lo leggo e lo analizzo
righe_csv = "prodotto;quantita;prezzo\nPane;3;1.20\nLatte;2;1.50\nUova;1;2.80"
with open("spesa.csv", "w", encoding="utf-8") as f:
    f.write(righe_csv)

# Leggo il CSV, salto l'intestazione, converto i numeri e calcolo i totali
totale_generale = 0
with open("spesa.csv", "r", encoding="utf-8") as f:
    intestazione = f.readline() # leggo e scarto la prima riga
    for riga in f:
        campi = riga.strip().split(";")
        prodotto = campi[0]
        quantita = int(campi[1])
        prezzo = float(campi[2])
        subtotale = quantita * prezzo
        totale_generale += subtotale
    print(f"{prodotto}: {quantita} x {prezzo:.2f} = {subtotale:.2f} euro")
print(f"TOTALE: {totale_generale:.2f} euro")
```

Questo è un esempio completo e realistico. Nota i passaggi chiave: `f.readline()` legge e scarta l'intestazione; `.strip().split(";")` divide la riga nei suoi campi; `int()` e `float()` convertono i campi numerici. Senza quelle conversioni, `quantita * prezzo` darebbe errore perché staresti moltiplicando del testo.

Mini esercizio

Crea un CSV con intestazione nome:voto e 5 studenti con i loro voti. Poi leggilo (saltando l'intestazione), converti i voti in numeri e stampa la media della classe con 2 decimali.

Checklist: hai imparato a...

- ☐ So cos'è un file CSV e come è fatto (righe e campi separati).
- ☐ So leggere un CSV come file di testo e separare i campi con `.split()`.
- ☐ So saltare la riga di intestazione quando serve.
- ☐ So convertire i campi numerici con `int()/float()` prima dei calcoli.

Capitolo 6

Pulire e trasformare i dati

I dati veri sono sporchi. Spazi di troppo, maiuscole a caso, virgole al posto dei punti, righe vuote. Prima di analizzare qualsiasi cosa, bisogna pulire. Questo lavoro, poco glamour ma fondamentale, occupa gran parte del tempo di chi lavora con i dati. La buona notizia: con le basi delle stringhe sai già farlo.

Gli strumenti principali li conosci: `.strip()` toglie gli spazi ai bordi, `.lower()` e `.upper()` uniformano le maiuscole, `.replace()` sostituisce parti di testo. Sono metodi delle stringhe, e si possono concatenare: `testo.strip().lower()` prima toglie gli spazi e poi mette tutto minuscolo.

Un caso classico in Italia: i numeri decimali scritti con la virgola invece del punto. Python vuole il punto. La soluzione è semplice: prima sostituisci la virgola con il punto con `.replace(",", ".")`, poi converti con `float()`. È un trucco che userai spessissimo con i dati italiani.

Un'altra trasformazione utile è validare i dati: controllare che un valore sia accettabile prima di usarlo. Per esempio, scartare le righe vuote, o ignorare un prezzo negativo. Combinando le condizioni (if) con la pulizia, rendi il tuo programma robusto: capace di gestire dati imperfetti senza bloccarsi.

Esempio pratico

```
# Dati sporchi tipici e come pulirli
dati_sporchi = [" Mela ", "BANANA", "kiwi", "", " Arancia"]

puliti = []
for valore in dati_sporchi:
    pulito = valore.strip()          # tolgo spazi ai bordi
    if pulito == "":                # scarto le righe vuote
        continue
    pulito = pulito.capitalize()     # prima lettera maiuscola, resto minuscolo
    puliti.append(pulito)
print("Dati puliti:", puliti)

# Prezzo italiano con la virgola -> numero
prezzo_italiano = "19,99"
prezzo = float(prezzo_italiano.replace(",", "."))
print(f"Prezzo utilizzabile nei calcoli: {prezzo:.2f}")
```

Il ciclo fa tre cose tipiche della pulizia: toglie gli spazi, scarta i valori vuoti (con `continue`, che salta al prossimo giro) e uniforma il formato con `capitalize()`. Il secondo blocco mostra il trucco della virgola: `.replace(",", ".")` prima di `float()`. Questi due schemi coprono la maggior parte delle pulizie che ti capiteranno.

Mini esercizio

Hai questa lista di città scritte male: `[" roma", "MILANO ", "", "napoli"]`. Puliscile (togli spazi, scarta i vuoti, prima lettera maiuscola) e mettile in una nuova lista. Poi prendi il prezzo `"1.234,50"` (formato italiano con migliaia) e prova a convertirlo: cosa serve fare?

Checklist: hai imparato a...

- ☐ So usare `.strip()`, `.lower()`, `.upper()`, `.capitalize()` per uniformare il testo.
- ☐ So scartare le righe vuote con una condizione e `continue`.
- ☐ So convertire un numero in formato italiano (virgola) sostituendo prima la virgola con il punto.
- ☐ Ho capito che pulire i dati è un passaggio necessario, non opzionale.

Capitolo 7

Analizzare i dati: contare, sommare, media, filtrare

Ora che sai rappresentare e pulire i dati, è il momento di analizzarli, cioè di rispondere a domande: quanti sono? Qual è il totale? La media? Quali soddisfano un certo criterio? Sono le quattro operazioni base dell'analisi dei dati, e le fai tutte con gli strumenti che già conosci.

Contare significa sapere quanti elementi ci sono: `len()` su una lista. Sommare significa totalizzare una colonna: l'accumulatore in un ciclo, o `sum()` su una lista di numeri. La media è la somma diviso il conteggio. Con queste tre operazioni rispondi già a moltissime domande.

Filtrare significa selezionare solo gli elementi che rispettano una condizione: i clienti di Roma, i voti sopra il 7, le spese sopra i 100 euro. Si fa con un ciclo e un `if` dentro: per ogni elemento, se la condizione è vera, lo tieni (lo aggiungi a una nuova lista o lo conti). È una delle operazioni più potenti e più usate.

Combinando filtrare e calcolare ottieni analisi sofisticate con poche righe: la spesa media dei soli clienti di Roma, il numero di studenti promossi, il prodotto più venduto. Questo capitolo è il cuore della guida: qui le basi diventano davvero analisi dei dati.

Esempio pratico

```
# Analisi completa su una tabella di clienti
clienti = [
    {"nome": "Anna", "citta": "Roma", "spesa": 120.50},
    {"nome": "Marco", "citta": "Milano", "spesa": 89.00},
    {"nome": "Sara", "citta": "Roma", "spesa": 230.75},
    {"nome": "Luca", "citta": "Milano", "spesa": 45.20},
    {"nome": "Elena", "citta": "Roma", "spesa": 99.99},
]

# Contare e sommare
print(f"Numero clienti: {len(clienti)}")
totale = sum(c["spesa"] for c in clienti)
print(f"Spesa totale: {totale:.2f} euro")
print(f"Spesa media: {totale / len(clienti):.2f} euro")

# Filtrare: solo i clienti di Roma
romani = [c for c in clienti if c["citta"] == "Roma"]
print(f"\nClienti di Roma: {len(romani)}")
spesa_roma = sum(c["spesa"] for c in romani)
print(f"Spesa media a Roma: {spesa_roma / len(romani):.2f} euro")

# Trovare il cliente che ha speso di più
top = max(clienti, key=lambda c: c["spesa"])
print(f"\nCliente top: {top['nome']} con {top['spesa']:.2f} euro")
```

Qui usiamo tre tecniche potenti. `sum(c["spesa"] for c in clienti)` somma una colonna in una riga sola. La list comprehension `[c for c in clienti if ...]` filtra in una riga. `max(..., key=...)` trova l'elemento con il valore più alto in una colonna. Sono scorciatoie eleganti, ma sotto fanno esattamente quello che faresti con un ciclo e un `if`: niente magia.

Mini esercizio

Usando la lista clienti dell'esempio, calcola: quanti clienti hanno speso più di 100 euro, e qual è la loro spesa totale. Poi trova il cliente che ha speso di meno (usa `min` con `key`).

Checklist: hai imparato a...

- ☐ So contare con `len()` e sommare una colonna con `sum()` o un accumulatore.
- ☐ So calcolare una media (somma diviso conteggio).
- ☐ So filtrare gli elementi che rispettano una condizione.
- ☐ So trovare il massimo o il minimo di una colonna con `max()/min()` e `key`.

Capitolo 8

Automazione: far fare a Python il lavoro ripetitivo

L'automazione è forse l'uso più immediatamente utile di Python nella vita reale. L'idea è semplice: qualsiasi compito noioso e ripetitivo che faresti a mano, Python lo fa per te, più velocemente e senza errori. Rinominare 200 file, mandare 50 email personalizzate, pulire un elenco, generare un report ogni settimana: tutto automatizzabile.

Il principio di base dell'automazione è il ciclo: invece di fare un'operazione una volta a mano, la descrivi una volta sola e la fai ripetere a Python su tutti gli elementi. Se sai scrivere un for, sai già automatizzare. La differenza tra fare una cosa e farne mille è, per Python, una riga.

Un'automazione tipica ha tre fasi: leggere i dati di partenza (da una lista, un file, un CSV), elaborarli (calcolare, trasformare, filtrare) e produrre un risultato (stampare un report, scrivere un nuovo file). Riconoscere queste tre fasi ti aiuta a strutturare qualsiasi piccolo automatismo.

Le funzioni sono l'alleato dell'automazione: una volta che hai scritto una funzione che fa un compito (per esempio: calcola il totale di una fattura), puoi richiamarla quante volte vuoi, su dati diversi, senza riscrivere il codice. Costruisci i tuoi strumenti una volta e li riusi sempre.

Esempio pratico

```
# Automazione: genero un report di spese da una lista, in automatico
spese = [
    {"voce": "Affitto", "importo": 600.00},
    {"voce": "Spesa", "importo": 240.50},
    {"voce": "Trasporti", "importo": 85.00},
    {"voce": "Bollette", "importo": 175.30},
]

BUDGET = 1200.00

# Fase 1: i dati ci sono. Fase 2: elaboro. Fase 3: produco il report.
totale = sum(s["importo"] for s in spese)
rimanente = BUDGET - totale

print("=== REPORT SPESE DEL MESE ===")
for s in spese:
    quota = (s["importo"] / totale) * 100
    print(f"{s['voce']:<12} {s['importo']:>8.2f} euro  ({quota:4.1f}%)"
print("-" * 35)
print(f"{'TOTALE':<12} {totale:>8.2f} euro")
print(f"{'BUDGET':<12} {BUDGET:>8.2f} euro")
print(f"{'RIMANENTE':<12} {rimanente:>8.2f} euro")
```

Questo è un vero piccolo strumento di automazione. Le tre fasi sono visibili: i dati (spese), l'elaborazione (totale, percentuali, rimanente) e il report stampato. Nota la formattazione allineata con :<12 (allinea a sinistra in 12 spazi) e :>8.2f (allinea a destra, 2 decimali): rende il report ordinato e professionale. Cambia i dati e il report si rigenera da solo: questa è l'automazione.

Mini esercizio

Crea una lista di 4 voci di entrate (voce, importo). Genera un report che stampa ogni entrata con la sua percentuale sul totale, e in fondo il totale delle entrate. Usa la formattazione allineata per renderlo ordinato.

Checklist: hai imparato a...

- ☐ Ho capito che automatizzare significa far ripetere a Python un compito su molti elementi.
- ☐ So riconoscere le tre fasi: leggere i dati, elaborarli, produrre il risultato.
- ☐ So generare un report leggibile con la formattazione allineata.
- ☐ Ho capito che le funzioni mi permettono di riusare il codice.

Capitolo 9

Funzioni e riuso: costruire i tuoi strumenti

Una funzione è un blocco di codice con un nome, che fa un compito preciso e che puoi richiamare quando vuoi. È lo strumento che trasforma il codice usa-e-getta in strumenti riutilizzabili. Quando lavori con dati e automazioni, le funzioni diventano indispensabili: ti permettono di non ripeterti e di organizzare il lavoro in pezzi chiari.

Una buona funzione fa una cosa sola e la fa bene. Riceve dei dati in ingresso (i parametri) e restituisce un risultato (con return). Per esempio, una funzione `calcola_iva(prezzo)` prende un prezzo e restituisce il prezzo con l'IVA. Una volta scritta, la usi su qualsiasi prezzo senza riscrivere la formula.

Il return è diverso dal print(). print() mostra qualcosa a schermo e basta. return restituisce un valore che puoi salvare in una variabile e riutilizzare in altri calcoli. Per costruire strumenti veri ti serve return: è la differenza tra una funzione che 'dice' e una che 'produce'.

Le funzioni possono avere parametri con valori di default, che le rendono flessibili: `calcola_iva(prezzo, aliquota=0.22)` usa il 22% se non specifichi altro, ma puoi passare un'aliquota diversa quando serve. Mettere insieme più funzioni che si richiamano a vicenda è il modo in cui si costruiscono programmi grandi partendo da pezzi piccoli e testati.

Esempio pratico

```
# Costruisco i miei strumenti riutilizzabili come funzioni
def calcola_iva(prezzo, aliquota=0.22):
    """Restituisce il prezzo con IVA."""
    return prezzo * (1 + aliquota)

def applica_sconto(prezzo, percentuale):
    """Restituisce il prezzo scontato."""
    return prezzo * (1 - percentuale / 100)

def prezzo_finale(prezzo_base, sconto_perc):
    """Combina sconto e IVA: prima sconta, poi aggiunge IVA."""
    scontato = applica_sconto(prezzo_base, sconto_perc)
    return calcola_iva(scontato)

# Uso i miei strumenti su prodotti diversi
for prezzo in [100, 50, 19.99]:
    finale = prezzo_finale(prezzo, 10) # 10% di sconto
    print(f"Base {prezzo:.2f} -> finale (sconto 10% + IVA): {finale:.2f} euro")
```

Guarda come le funzioni si combinano: `prezzo_finale` richiama `applica_sconto` e `calcola_iva`. Ogni funzione fa una cosa sola, e insieme risolvono un problema complesso. Le ho scritte una volta e le ho usate su tre prezzi diversi con un ciclo. Questo è il riuso: scrivi lo strumento una volta, lo usi sempre. Le frasi tra triple virgolette sono docstring: spiegano cosa fa la funzione.

Mini esercizio

Scrivi una funzione `media(lista_numeri)` che restituisce la media di una lista. Scrivi poi una funzione `voto_in lettere(voto)` che restituisce A, B, C o Bocciato. Usa entrambe su una lista di voti: stampa la media della classe e la lettera corrispondente.

Checklist: hai imparato a...

- ☐ So scrivere una funzione con parametri e return.
- ☐ Ho capito la differenza tra return (produce un valore) e print (mostra e basta).
- ☐ So usare parametri con valore di default.
- ☐ So combinare più funzioni per risolvere un problema più grande.

Capitolo 10

Gestire gli errori nei dati reali

Con i dati veri, gli errori sono la norma, non l'eccezione. Un campo vuoto, un testo dove ti aspettavi un numero, un file che non esiste. Un programma robusto non si blocca al primo intoppo: lo gestisce con eleganza e va avanti. Questo capitolo ti insegna a difendere il tuo codice dai dati imperfetti.

Lo strumento principale è try/except. Metti dentro try il codice che potrebbe fallire, e dentro except cosa fare se fallisce. Così, invece di un crash con un messaggio incomprensibile, il tuo programma stampa un avviso chiaro e continua. È la differenza tra un programma fragile e uno professionale.

Conviene catturare errori specifici. ValueError capita quando provi a convertire in numero qualcosa che non lo è (per esempio int("ciao")). ZeroDivisionError capita quando dividi per zero. Catturando l'errore giusto, dai messaggi precisi e non nascondi problemi diversi che invece vorresti vedere.

Nei dati reali, la strategia tipica è: provo a elaborare ogni riga dentro un try; se una riga è rotta, la salto con un avviso e continuo con le altre. Così una sola riga sbagliata su mille non ti rovina l'intera analisi. È un atteggiamento che ti distingue subito come persona che sa lavorare con i dati veri.

Esempio pratico

```
# Elaboro dati imperfetti senza che il programma si blocchi
righe = ["Pane;3;1.20", "Latte;due;1.50", "Uova;1;2.80", "Riga;rotta", "Olio;2;4.50"]

totale = 0
righe_ok = 0
for numero, riga in enumerate(righe, 1):
    try:
        campi = riga.split(";")
        quantita = int(campi[1])
        prezzo = float(campi[2])
        subtotale = quantita * prezzo
        totale += subtotale
        righe_ok += 1
        print(f"Riga {numero}: OK -> {subtotale:.2f} euro")
    except (ValueError, IndexError):
        print(f"Riga {numero}: SCARTATA (dati non validi)")

print(f"\nRighe valide: {righe_ok} su {len(righe)}")
print(f"Totale (solo righe valide): {totale:.2f} euro")
```

Questo esempio è realistico: su 5 righe, due sono rotte ("due" non è un numero, e "Riga;rotta" non ha abbastanza campi). Senza try/except il programma si bloccherebbe alla seconda riga. Invece le scarta con un avviso e calcola il totale sulle righe valide. Cattura due errori insieme: ValueError (conversione fallita) e IndexError (campo mancante). Questo è codice che sa gestire il mondo reale.

Mini esercizio

Hai una lista di età inserite come testo: ["25", "trenta", "40", "", "18"]. Convertile in numeri saltando quelle non valide con try/except, e stampa quante età valide hai trovato e la loro media.

Checklist: hai imparato a...

- ☐ So usare try/except per evitare che il programma si blocchi.
- ☐ So catturare errori specifici come ValueError e ZeroDivisionError.
- ☐ So saltare una riga di dati rotta e continuare con le altre.
- ☐ Ho capito che gestire gli errori è essenziale con i dati reali.

Capitolo 11

Python e l'AI: usare l'intelligenza artificiale nel tuo lavoro

Eccoci al collegamento con l'intelligenza artificiale. Sgombriamo subito il campo: in questo capitolo non costruirai un'AI (serve ben altro). Imparerai una cosa più utile e alla tua portata: come Python e l'AI lavorano insieme, e come le basi che hai ti rendono capace di usare l'AI con consapevolezza.

Oggi l'AI generativa (come ChatGPT) è bravissima a scrivere codice Python. Questo cambia il tuo ruolo: non devi ricordare tutto a memoria, devi saper chiedere bene, leggere quello che l'AI produce, capirlo, provarlo e correggerlo. Le basi che hai imparato servono esattamente a questo: senza, non potresti giudicare se il codice dell'AI è giusto o sbagliato.

Un uso concreto e potente è l'AI come assistente per i dati: le descrivi i tuoi dati e cosa vuoi ottenere, lei ti suggerisce il codice, tu lo verifichi e lo adatti. Per esempio: 'Ho un CSV con colonne prodotto, quantità, prezzo. Scrivimi un codice base che calcola il totale per prodotto.' L'AI ti dà una bozza, tu la capisci e la fai tua.

La regola d'oro, che trovi approfondita nella guida 'AI senza illusioni', è sempre la stessa: prima provi tu, poi chiedi; e non usare mai codice che non capisci. L'AI è un acceleratore straordinario per chi ha le basi, e una trappola per chi non le ha. Tu, arrivato qui, le basi le hai: usala come un collega esperto a cui chiedere consiglio, non come un oracolo da credere sulla parola.

Esempio pratico

```
# Esempio: codice "suggerito dall'AI" che tu sai leggere, verificare e usare
# Richiesta all'AI: "raggruppa le quantità vendute per prodotto"

vendite = [
    {"prodotto": "Pane", "quantita": 3},
    {"prodotto": "Latte", "quantita": 2},
    {"prodotto": "Pane", "quantita": 5},
    {"prodotto": "Uova", "quantita": 1},
    {"prodotto": "Latte", "quantita": 4},
]

# L'AI propone un dizionario come "registro dei totali": tu lo capisci e lo verifichi
totali = {}
for v in vendite:
    prodotto = v["prodotto"]
    if prodotto in totali:
        totali[prodotto] += v["quantita"]
    else:
        totali[prodotto] = v["quantita"]

for prodotto, quantita in totali.items():
    print(f"{prodotto}: {quantita} venduti")
```

Questo è un codice che l'AI ti darebbe per 'raggruppare per prodotto'. La cosa importante non è che l'abbia scritto l'AI: è che TU lo capisci. Riconosci il dizionario usato come registro, il ciclo, la condizione if/else che controlla se il prodotto è già presente. Proprio perché conosci queste basi, puoi verificare che il risultato sia giusto (Pane 8, Latte 6, Uova 1) e fidarti del codice. Senza basi, dovresti solo sperare.

Mini esercizio

Prendi l'esempio del raggruppamento e adattalo: invece delle quantità, raggruppa la SPESA totale per città (usa la lista clienti del capitolo 7). Poi scrivi il prompt che useresti per chiedere all'AI di spiegarti questo codice riga per riga.

Checklist: hai imparato a...

- ☐ Ho capito che non costruisco l'AI, ma imparo a lavorarci insieme.
- ☐ So che le basi mi servono per leggere e verificare il codice che l'AI produce.
- ☐ So usare l'AI come assistente per i dati, verificando sempre il risultato.

☐ Conosco la regola d'oro: prima provo io, e non uso codice che non capisco.

Capitolo 12

Capstone: dal dato grezzo al mini-report

Sei arrivato all'ultimo capitolo. Ora mettiamo insieme tutto quello che hai imparato in un unico progetto completo, che parte da dati grezzi in un file CSV e arriva a un report ordinato. Questo è il tipo di programma che, in piccolo, fanno gli analisti di dati ogni giorno. E tu sei in grado di scriverlo.

Il progetto unisce tutte le competenze della guida: leggere un CSV (cap. 5), pulire e convertire i dati (cap. 6), gestire le righe rotte con try/except (cap. 10), organizzare il codice in funzioni (cap. 9), analizzare con conteggi, somme, medie e raggruppamenti (cap. 7), e produrre un report leggibile (cap. 8). Vedrai come i pezzi si incastrano.

Affrontare un progetto così può sembrare difficile, ma il segreto è scomporlo: non pensare 'devo fare tutto', pensa 'prima leggo i dati, poi li pulisco, poi li analizzo, poi stampo'. Un passo alla volta. È esattamente il modo di pensare strutturato di cui parlavamo nel primo capitolo: il vero superpotere che hai acquisito.

Quando avrai fatto girare questo progetto e lo avrai capito, potrai dire di saper trasformare dei dati grezzi in informazioni utili con Python. Non è più 'studiare la teoria': è saper fare. Da qui, la Roadmap del kit ti mostra come proseguire verso strumenti più potenti.

Esempio pratico

```
# CAPSTONE: analisi completa delle vendite da un CSV

# --- Preparo un CSV di esempio (nella realtà lo avresti già) ---
dati = "prodotto;categoria;quantita;prezzo\n" \
      "Pane;Alimentari;3;1.20\n" \
      "Quaderno;Cartoleria;5;2.50\n" \
      "Latte;Alimentari;2;1.50\n" \
      "Penna;Cartoleria;10;0.90\n" \
      "Riga;rotta\n" \
      "Uova;Alimentari;1;2.80"
with open("vendite.csv", "w", encoding="utf-8") as f:
    f.write(dati)

# --- Funzioni-strumento ---
def leggi_vendite(nomefile):
    """Legge il CSV, salta righe rotte, restituisce una lista di dizionari."""
    risultato = []
    with open(nomefile, "r", encoding="utf-8") as f:
        f.readline() # salto intestazione
        for riga in f:
            try:
                p, cat, q, pr = riga.strip().split(";")
                risultato.append({
                    "prodotto": p, "categoria": cat,
                    "quantita": int(q), "incasso": int(q) * float(pr)
                })
            except (ValueError, IndexError):
                print(f" [avviso] riga scartata: {riga.strip()}")
    return risultato

def totali_per_categoria(vendite):
    """Raggruppa l'incasso per categoria."""
    tot = {}
    for v in vendite:
        tot[v["categoria"]] = tot.get(v["categoria"], 0) + v["incasso"]
    return tot

# --- Programma principale ---
vendite = leggi_vendite("vendite.csv")
incasso_totale = sum(v["incasso"] for v in vendite)

print("=== REPORT VENDITE ===")
```

```

print(f"Prodotti validi analizzati: {len(vendite)}")
print(f"Incasso totale: {incasso_totale:.2f} euro")
print(f"Incasso medio per prodotto: {incasso_totale / len(vendite):.2f} euro\n")

print("Incasso per categoria:")
for categoria, incasso in totali_per_categoria(vendite).items():
    quota = incasso / incasso_totale * 100
    print(f" {categoria:<12} {incasso:>7.2f} euro ({quota:4.1f}%)")

top = max(vendite, key=lambda v: v["incasso"])
print(f"\nProdotto con incasso più alto: {top['prodotto']} ({top['incasso']:.2f} euro)")

```

Questo è un programma completo e realistico, e usa TUTTO quello che hai imparato. Leggi il CSV con una funzione che gestisce le righe rotte (vedi l'avviso sulla riga 'rotta'). Raggruppi per categoria con un'altra funzione. Calcoli totale, media, percentuali e trovi il top. Se cambi il CSV, il report si rigenera. Studialo riga per riga: se lo capisci tutto, hai completato il percorso di questa guida.

Mini esercizio

Estendi il capstone: aggiungi una funzione `prodotto_piu_venduto(vendite)` che restituisce il prodotto con la quantità totale più alta (non l'incasso). Stampalo nel report. Suggerimento: raggruppa le quantità per prodotto come nel capitolo 11.

Checklist: hai imparato a...

- ☐ Ho eseguito il capstone e ottenuto il report completo.
- ☐ Riconosco nel progetto tutte le tecniche dei capitoli precedenti.
- ☐ So scomporre un problema grande in passi piccoli (leggere, pulire, analizzare, stampare).
- ☐ Ho esteso il progetto con una mia funzione: so costruire, non solo copiare.

E adesso?

Se sei arrivato qui e hai fatto gli esercizi, hai trasformato le basi di Python in qualcosa di concreto: sai leggere dati da un file, pulirli, analizzarli, automatizzare un compito e collaborare con l'AI in modo consapevole. Non è poco: è esattamente il punto di partenza di chi lavora con i dati.

Il passo successivo è nella Roadmap del kit (cartella 09): ti mostra i percorsi possibili verso l'automazione avanzata, l'analisi dei dati con strumenti dedicati, l'AI generativa e il reporting professionale. Hai costruito le fondamenta. Ora puoi scegliere dove andare.

Nessuno nasce imparato - ma tu, un capitolo alla volta, hai imparato.